In GNU Bison, the special symbol `error` is a built-in **nonterminal** used for **error recovery** in grammars. It’s not something you define—it’s provided by Bison to help your parser recover from syntax errors instead of just aborting.

🧠 What `error` actually is

* `error` is a **reserved token-like symbol** that Bison injects when it detects a syntax error.
* You can include it in your grammar rules to say: *“If a syntax error happens here, recover in this way.”*

🔧 Basic idea

When Bison encounters invalid input:

1. It reports a syntax error (calls `yyerror`).
2. It tries to **recover** by:

   * Popping states from the parser stack until it finds a rule that can accept `error`.
   * Shifting the `error` token.
   * Then continuing parsing from there.

📌 Example

```bison
stmt:
    expr ';'
  | error ';'   { yyerrok; }
;
```

What this means:

* Normally: parse `expr ;`
* If something goes wrong while parsing `expr`:

  * Bison inserts `error`
  * It skips input until it finds `;`
  * Then reduces using `error ';'`
  * Executes `{ yyerrok; }` to reset error state

🔁 Key macros used with `error`

`yyerrok`

* Marks the error as **handled**
* Allows future errors to be reported normally

`yyclearin`

* Discards the current lookahead token
* Useful if the current token is causing trouble

🔄 How recovery works step-by-step

Suppose input is:

```
x + * y;
```

And grammar expects:

```
expr: expr '+' expr | IDENTIFIER
```

* Parser hits `*` where it doesn't belong → syntax error
* Bison:

  * Pops stack until it finds a rule that accepts `error`
  * If you have `stmt: error ';'`, it:

    * Skips tokens until `;`
    * Continues parsing after that

⚠️ Important behaviors

1. `error` is not a normal token

* You **don’t return it from the lexer**
* Bison generates it internally

---

2. It only works where you explicitly allow it

If you don’t include `error` in your grammar rules, Bison:

* Will just abort parsing on error

---

3. It helps prevent cascading errors

Without recovery:

* One syntax error → dozens of confusing follow-up errors

With `error` rules:

* Parser resynchronizes at safe points (like `;` or `}`)

🧩 Common recovery patterns

Statement-level recovery

```bison
stmt:
    expr ';'
  | error ';' { yyerrok; }
;
```

Block-level recovery

```bison
block:
    '{' stmt_list '}'
  | '{' error '}' { yyerrok; }
;
```

Skip to newline (useful in REPLs)

```bison
line:
    expr '\n'
  | error '\n' { yyerrok; }
;
```

🧠 Mental model

Think of `error` as:

> “A placeholder symbol Bison uses when parsing fails, allowing you to define where and how to resume.”

🚫 Common mistakes

* ❌ Expecting `error` to come from the lexer
* ❌ Forgetting `yyerrok` → parser stays in error state
* ❌ Not placing recovery points → poor recovery

If you want, I can show a full working mini Bison + Flex example with error recovery so you can see it in action.

Bison’s error recovery is a two-part process: you **teach the grammar where recovery is allowed** by mentioning the built-in token `error`, and then Bison’s runtime **repairs the parse stack and input stream** until it can continue from one of those recovery points. `error` is always

defined, you do not declare it, and Bison generates it internally when a syntax error occurs. ([GNU][1])

The algorithm, step by step, is:

1. A syntax error is detected. Bison reports it, unless it is already in recovery mode. While recovering, syntax-error diagnostics are suppressed. ([GNU][1])
2. Bison pops parser states and their semantic values off the stack until it finds a state where shifting `error` is valid. This discards the already-parsed context back to the nearest recovery point your grammar allows. ([GNU][1])
3. Bison shifts the synthetic `error` token. This makes the recovery rule viable, such as `stmt: error ';'`. ([GNU][1])
4. Bison then checks the current lookahead token. If that token cannot follow the recovery rule, it discards input tokens until it finds one that can. In the common `error ';'` pattern, it keeps discarding until `;` appears. ([GNU][1])
5. Once a valid token is found, parsing resumes normally from the recovery rule. ([GNU][1])

A concrete example from the manual is:

```bison
stmts:
  %empty
| stmts '\n'
| stmts exp '\n'
| stmts error '\n'
;
```

If an error happens in the middle of `exp`, Bison pops back to the last state where `error` is acceptable, then discards input until the next newline so that `stmts error '\n'` can reduce successfully. ([GNU][1])

Two helper macros matter a lot. `yyerrok` ends the recovery period immediately, so future syntax errors are reported right away. `yyclearin` discards the current lookahead token if the token already fetched after the error should not be re-used. ([GNU][1])

Bison also suppresses follow-up error messages during recovery: after a syntax error, it waits until **three consecutive input tokens have been successfully shifted** before it resumes normal error reporting. That is what prevents one bad token from producing a long cascade of messages. ([GNU][1])

The key design choice is where you place `error` in the grammar. Typical recovery points are “skip to semicolon” (`stmt: error ';'`) or “recover at a matching close delimiter” (`primary: '(' error ')'`). The manual explicitly recommends those as useful strategies. ([GNU][1])

One practical caution: tokens and stack objects that get discarded during recovery may own memory, so you need to make sure your semantic-value destructors or cleanup logic handle discarded symbols correctly. The manual calls this out as a possible source of memory leaks. ([GNU][1])

If you want, I can also show the same algorithm as a small state-machine diagram or walk through one real parse trace token by token.

[1]: https://www.gnu.org/software/bison/manual/html_node/Error-Recovery.html "Error Recovery (Bison 3.8.1)"

Alright—here’s the **full, precise, operational view** of Bison’s error recovery algorithm, not just the intuition. I’ll walk it like the parser actually executes it.

🧠 Big picture

Bison is an **LALR(1) shift-reduce parser**. Error recovery is triggered when:

> The current parser state has **no valid action** for the current lookahead token.

At that moment, Bison enters **error recovery mode** and runs a specific algorithm.

⚙️ The algorithm (step-by-step)

1. Syntax error is detected

Condition:

* Parser state = `S`
* Lookahead token = `t`
* Parse table has **no shift or reduce action for (S, t)**

What happens:

* `yyerror(...)` is called (unless already recovering)
* Internal flag `yyerrstatus` is set (typically to 3)

---

2. If already recovering, suppress repeated errors

If `yyerrstatus > 0`:

* Bison **does NOT call `yyerror` again**
* This prevents cascading error spam

---

3. Try to recover by stack unwinding

Now the core loop:

> Pop states until one can shift `error`

Formally:

```
while (true):
  if current_state accepts 'error':
      break
  else:
```

```
      pop stack (state + semantic value)
      if stack empty:
          abort parse (failure)
```

Important:

* This **destroys partially parsed constructs**
* You lose semantic values unless cleaned up

---

4. Shift the special `error` token

Once a suitable state is found:

* Bison performs a **shift of the synthetic token `error`**
* This transitions to a new state as defined by the grammar

Now the parser is aligned with a rule like:

```bison
stmt: error ';'
```

---

5. Discard invalid lookahead tokens

Now Bison must find a token that can follow `error`.

If current lookahead token is invalid:

```
while (lookahead token cannot be shifted):
    discard lookahead token
    read next token
```

Key effect:

* Input is **skipped forward** until a synchronization point

---

6. Resume parsing

Once a valid token is found:

* Parsing continues normally
* Eventually a rule containing `error` is reduced

Example:

```bison
stmt: error ';'
```

When `;` is found:

* rule reduces
* action runs

---

7. Error recovery mode behavior

After recovery begins:

* `yyerrstatus = 3`
* Each successful **shift** decrements it:

  ```
  yyerrstatus--
  ```

While `yyerrstatus > 0`:

* No new error messages are reported

👉 This enforces:

> “3 successful shifts before reporting new errors”

---

8. Optional user control

`yyerrok;`

* Sets `yyerrstatus = 0`
* Ends recovery mode immediately

`yyclearin;`

* Discards current lookahead token
* Forces parser to read a fresh one

🔄 Concrete example

Grammar:

```bison
stmt:
    expr ';'
  | error ';' { yyerrok; }
;
```

Input:

```
x + * y ;
```

Execution trace

1. Parse `x +`
2. See `*` → invalid → syntax error

Recovery:

Step A: stack popping

* Pop until state accepts `error`
* Likely back to `stmt`

Step B: shift `error`

* Now parsing `error ';'`

Step C: discard tokens

* `*` → discard
* `y` → discard
* `;` → acceptable

Step D: reduce

* `error ';'` reduces to `stmt`
* `{ yyerrok; }` resets error state

Step E: continue parsing

🧩 Important subtleties

1. `error` is not a lexer token

* Never returned by `yylex`
* Injected internally

---

2. Recovery points are grammar-defined

If you don’t write rules with `error`:

* Parser will just fail

---

3. Stack popping is aggressive

* Can remove large parts of parse tree
* May skip nested constructs

---

4. Token discarding is greedy

* Continues until a valid continuation is found
* This is why you usually sync on:

  * `;`
  * `}`
  * `\n`

---

5. Memory management matters

Discarded symbols:

* Do **not** run normal reductions
* You must use `%destructor` if needed

🧠 Mental model (precise)

Error recovery is:

> “Climb up the parse stack to a state that can accept `error`, then fast-forward the input until a token fits, and continue as if nothing happened.”

📊 Summary (algorithm form)

```
on syntax error:
  if not recovering:
    report error

  set yyerrstatus = 3

  // find recovery state
  while stack not empty:
    if state accepts 'error':
      shift 'error'
      break
    else:
      pop stack

  if stack empty:
    abort

  // discard input
  while lookahead invalid:
    discard token
    read next
```

```
  resume parsing
```

If you want, I can go one level deeper and show:

* the exact LR parsing table perspective
* or the actual C code structure Bison generates (`yyparse`) for this logic

Alright—let’s go **deep and precise**, closer to how Bison actually implements it internally.

I’ll walk you through the **actual recovery algorithm**, not just the intuition.

🧠 High-level structure

When a syntax error occurs, Bison does **three things in sequence**:

1. **Enter error state**
2. **Repair the parser stack (pop until `error` is acceptable)**
3. **Repair the input stream (discard tokens until parsing can continue)**

🔧 Step-by-step algorithm (precise)

Assume we are in the LR parsing loop and hit an error.

---

1. Detect syntax error

This happens when:

* current state = `S`
* lookahead token = `a`
* **no valid action** exists in the parse table for `(S, a)`

👉 At this point:

* `yyerror(...)` is called (unless already recovering)
* parser enters **error recovery mode**

---

2. If already recovering → suppress error messages

Bison uses an internal counter (`yyerrstatus`):

* If `yyerrstatus > 0`, it means:

  > “we are already recovering”

👉 In this case:

* **do NOT call `yyerror` again**
* prevents cascades of errors

---

3. Try to shift the special token `error`

Now comes the core logic.

Bison tries to **force the grammar to accept an `error` token**.

Algorithm:

```
loop:
  if current state can shift `error`:
      shift `error`
      goto step 4
  else:
      pop one symbol from stack
      if stack becomes empty:
          abort parsing
      repeat loop
```

👉 This is the **stack repair phase**

What “can shift error” means

In LR terms:

* There exists a parser action:

  ```
  ACTION[state, error] = shift S'
  ```

This only happens if your grammar contains a rule involving `error`, e.g.:

```bison
stmt: error ';'
```

---

4. Shift `error` token

Once a valid state is found:

* Bison **pretends it read the token `error`**
* pushes it onto the stack
* transitions to a new state

👉 No lexer involvement—this is synthetic

---

5. Set recovery mode

Bison sets:

```c
yyerrstatus = 3;
```

Meaning:

> “We need to successfully shift 3 tokens before leaving recovery mode.”

---

6. Handle the lookahead token

Now Bison considers the **current lookahead token** (the one that caused the error).

Case A: token is acceptable

If the current state **can accept the lookahead token**:

👉 parsing continues normally

Case B: token is NOT acceptable

Then:

```
discard lookahead token
read next token
repeat until acceptable token found
```

👉 This is the **input repair phase**

---

7. Resume normal parsing

Once:

* a valid token is found
* parsing continues

Then:

* each successful shift decrements `yyerrstatus`

When:

```
yyerrstatus == 0
```

👉 error recovery mode ends
👉 new errors will be reported again

🔁 Full algorithm (compact pseudocode)

```text
on syntax error:

  if yyerrstatus == 0:
      call yyerror()

  yyerrstatus = 3

  # Stack repair
  while stack not empty:
      if ACTION[top_state, error] exists:
          shift(error)
          break
      else:
          pop stack

  if stack empty:
      abort

  # Input repair
  while lookahead token not acceptable:
      discard lookahead
      read next token
      if EOF:
          abort

  resume parsing
```

📦 Example walkthrough

Grammar:

```bison
stmt:
    expr ';'
  | error ';'
;
```

Input:

```
x + * y ;
```

What happens:

1. Parser sees `*` where invalid → error

2. Stack pop:

   * Pops until it reaches a state where `error` fits into `stmt`

3. Shift `error`

4. Now expects `';'`

5. Discards tokens:

   * `*`, `y`
   * stops at `;`

6. Reduces:

   ```
   stmt → error ';'
   ```

7. Continues parsing next input

⚙️ Subtleties (important)

🔷 1. `error` behaves like a terminal

* Not returned by lexer
* Only exists inside parser

🔷 2. Recovery depends entirely on your grammar

If you don’t write rules like:

```bison
stmt: error ';'
```

👉 Bison has **nowhere to recover to**
👉 it will likely abort

🔷 3. Lookahead is reused unless discarded

* Bison does **not automatically discard the first bad token**
* Only discards if it cannot proceed

🔷 4. `yyclearin`

You can force discard:

```c
yyclearin;
```

👉 throws away current lookahead immediately

🔷 5. `yyerrok`

```c
yyerrok;
```

* sets `yyerrstatus = 0`
* ends recovery early

🔷 6. Memory management issue

When popping stack or discarding tokens:

👉 semantic values are lost

If they allocate memory:

* you must define destructors (`%destructor`)

🧠 Mental model (accurate)

Bison recovery is:

> “Backtrack the parser stack to a known safe grammar point, inject an `error` token, then skip input until parsing can legally continue.”

⚠️ Why it works well

* Prevents cascading errors
* Lets you define **synchronization points** (`;`, `}`, etc.)
* Keeps parser deterministic (still LR)

If you want, I can go one level deeper and show:

* the exact LR table entries involved (`ACTION` / `GOTO`)
* or how this behaves differently in GLR mode